

Airframe analysis & solution, v3

Re-checked on the full recordings (8 sensors × 30 minutes, 1.1 million frames), plus the real-time design, the AI fix agent (Part 2) and the self-learning error catalog (Part 3).

25 September 2026 · Every number in this report is recomputed by `verify_findings.py` (36/36 checks pass). Device addresses and network names are pseudonymised: L-nn laptops, H-nn phones/handhelds, I-nn IoT devices, APnn access points, NET-C corporate, NET-T tools.

The answer in one paragraph

For the whole 30 minutes, not one device could log in to the corporate Wi-Fi. 63 devices (30 laptops, 33 phones) tried 654 times on 26 access points across all 8 channels. Every time, the access point asks "who are you?", nothing comes back from the network side, it asks again after 30 s, and after 60 s it throws the device out ("802.1X authentication failed"). Devices keep retrying, waiting longer each time. After 3 to 5 failures, 18 laptops are put on a fast-reject list and bounced within seconds of every attempt. Meanwhile the tools network on the same radios works perfectly. **This is one site-wide problem in the login service behind the Wi-Fi (RADIUS or the path to it), not a radio problem.** It creates a probe storm that wastes airtime for everyone. A real-time version of our tool would flag it 36 seconds after the first failed login.

0 / 654 corporate log-in attempts that got past the identity step	63 corporate devices offline all 30 min (30 laptops, 33 phones)	18 laptops switched to fast- reject (1,477 bounce frames)	31 / 31 tools-network handshakes where the AP accepted the device's key (M3)
217/s peak probe replies; 31% re-sent (no acknowledgement)	8 devices whose access point never offers their network	36 s time to detect in real time (measured by replay)	≈15× laptop headroom over live traffic (9,552 frames/s)

Findings, ranked

#	Finding	Severity	Confidence	Where
F1	Corporate 802.1X login stalls at "who are you?" for every device, all 30 min	Critical	High	Site: 26 APs, 8 channels, 8 sensors
F2	Join → kick → wait → retry loop, 8–14 cycles per device, waits grow from ~28 s to ~200 s	High	High	All 63 corporate devices
F3	New: 18 laptops moved to fast-reject (reason 2 every 11–26 s)	High	High (pattern) · Med (why)	18 APs, from 6.7 to 10.7 min
F4	Probe storm with 31% unacknowledged replies	High	High	All channels; peak minute 3
F5	7 networks missing on 6 APs (AP13 silent); 8 devices can never join	Medium	Medium	AP03, 06, 13, 16, 17, 28

F6	Beacons at 1 Mbit/s (impossible on 5 GHz) → 10–19% airtime if real	Medium	Low	Worst on ch 36, 40
F7	Sensors hear access points far better than devices	Info	High	All sensors
F8	15 devices said "I'm leaving" within 9 s at the start	Watch	High	6 sensors
F9	Tools network (IoT) healthy; no roams, no rejects anywhere	Healthy	High	24 IoT devices

1 What changed when we looked at all 30 minutes

v2 was built on the first 50 seconds of each recording. We converted the full recordings and re-tested every claim. The main story held; the scale, several numbers and one prediction did not.

v2 said (50 s)	v3 verified (30 min)	Verdict
Recordings are ~33 min; JSON holds 2.5%	Exactly 30.0 min (1,802 s) each; old JSON held 2.8%	Corrected
16 laptops fail at the identity step	63 devices (30 laptops + 33 phones), 654 attempts, 0 past the identity step, 26 APs	Much bigger
Kick at 60 s (seen twice, outside the window)	506 kicks with reason 23; 488 of 489 measured exactly 60.0 s after join; re-ask 30.0 s	Confirmed
Loop: ~30 cycles per laptop predicted	8–14 cycles (median 10): devices wait longer each time (28 s → ~200 s)	Prediction wrong
(not visible)	18 laptops switch to fast-reject (reason 2) after 3–5 failures	New
4 laptops have no network to join	8 devices (4 laptops, 4 phones); networks stay missing all 30 min	Bigger
Probe storm ~56 replies/s, 32% re-sent	Mean 96.5/s, peak 217/s (minute 3), 31.3% re-sent every minute	Bigger
IoT devices send encrypted data	Only 4 of 24 are <i>heard</i> sending; the rest are proven connected by M3 (sensor cannot hear their uplink)	Corrected wording
Beacons missed ~2%	1.7–2.3% on all 53 networks; longest gap 0.41 s; no sensor ever went deaf	Confirmed
"1 Mbit/s rate may be a parsing error"	Wireshark reads the same 1 Mbit/s from the radio header: it is in the data	Confirmed

2 How everything was verified

- **Two independent decoders agree.** Your scapy converter and Wireshark's tshark were compared on all 28,536 overlapping frames: every field matches (time, rate, channel, signal, type, flags, addresses, sequence, status and reason codes, EAP, EAPOL message numbers). The only difference is by design: tshark reports the on-air length of the 171 truncated data frames.
- **Security types read from the air, not guessed.** Beacons advertise NET-C with AKM 1 (802.1X) and NET-T with AKM 2 (pre-shared key).
- **Codes checked against the standard.** All reason, status and EAP meanings come from Wireshark 4.6.9's tables (IEEE 802.11-2020 and later, RFC 3748), saved in `catalog/ieee_codes.json`.

- **Every quoted number is re-computed** by `verify_findings.py` : 36/36 checks pass. My draft evidence had three wrong numbers (22 APs, 1.01 M beacons, 5 IoT devices); the checks caught them, and they are corrected to 26, 914,057 and 4.
- **Real time was measured, not estimated.** `realtime_bench.py` replays the real recordings into live tshark pipes (see section 4).

Signs this is an emulated scenario: every device's first attempt falls on an exact 2-second schedule; signal strength never changes; 5 GHz beacons at 1 Mbit/s. Protocol logic transfers to real sites; airtime maths from this data does not.

3 What is wrong, and where

Critical F1. Corporate login never gets past "who are you?"

Seen: each attempt goes: knock (accepted) → sign in (accepted) → the access point sends "EAP Identity request" → no next question, no success, no failure, no key messages. The access point re-asks every **30.0 s** and kicks the device at **60.0 s** with reason 23 ("802.1X authentication failed"). 654 attempts, 63 devices, 26 access points, 8 channels, 0 successes.

Why the network side: for the 12 devices the sensors can hear, the device answers its identity within 1 ms, every time; the next step belongs to the access point and the login server, and it never comes. On the tools network, same radios, the access point accepts the device's key (M3) in 31 of 31 handshakes seen. A radio or device fault would not hit 26 APs and two device types identically.

Most likely causes, in the order an engineer should check: RADIUS/NPS server down or unreachable from the controller; wrong shared secret after a change; firewall/VLAN change on the controller-to-RADIUS path; RADIUS overloaded. Headers alone cannot tell these apart; the RADIUS and controller logs can.

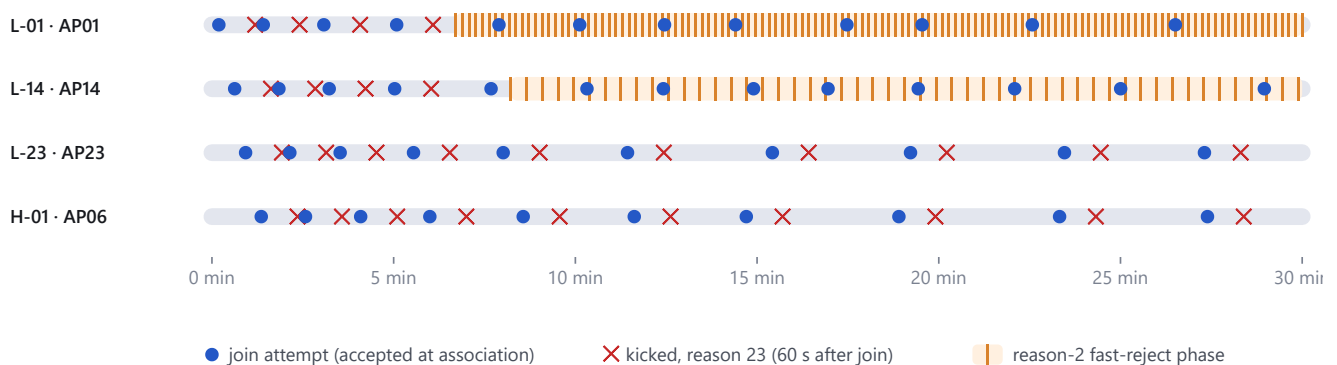


Figure 1. Four real devices over 30 minutes. L-01 and L-14 go through four slow kicks, then (orange) the network switches them to fast-reject. L-23 and phone H-01 stay in the slow 60-second cycle all the way. Attempts get further apart over time because devices back off.

High F2. The retry loop: devices spend the whole 30 minutes failing

Each corporate device made 8–14 attempts (median 10). The wait between a kick and the next try grows from a median of 28 s in the first 5 minutes to ~200 s after 20 minutes, so devices are offline and, over time, try less and less. This is the "death loop" from the brief; the tool links it to F1 instead of reporting 63 separate problems.

High F3. New: 18 laptops are put on a fast-reject list

After 3–5 slow kicks, 18 laptops (each on its own access point, all among APs 1–22) switch to a different treatment: the access point sends "previous authentication no longer valid" (reason 2) within a median of 8 s of every rejoin, and keeps repeating it every 11–15 s (as a deauth) or ~26 s (as a disassociation). 1,477 such frames. It starts one laptop at a time, about every 15 s, from 6.7 to 10.7 minutes; none of these laptops gets a reason-23 kick after its switch. The frames carry the access point's own running sequence numbers, so they are genuine, not spoofed.

Plant meaning: this matches a controller "client exclusion / block-list after repeated 802.1X failures" policy. **Important for the fix:** repairing RADIUS will not bring these 18 laptops back until their exclusion is cleared or times out. Why only these 18 (and not the other 45 devices with more failures) cannot be seen in headers; it is a question for the controller configuration.

High F4. Probe storm: devices search non-stop, a third of replies are lost

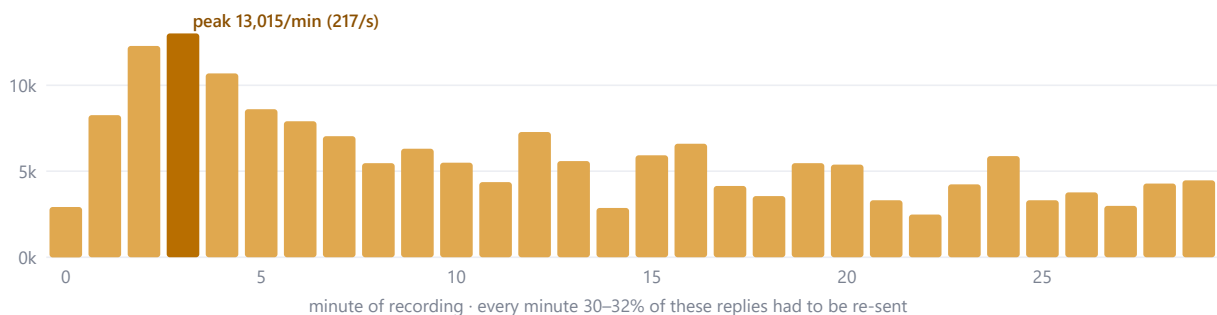


Figure 2. Access-point replies to device scans, per minute. The storm peaks while the most devices are failing, then eases as they back off.

173,964 probe replies in 30 minutes (mean 96.5/s, peak 217/s). 31.3% had to be re-sent because the device did not acknowledge, usually because it had already moved on to scan the next channel. Almost every retry in the data (54,484 of 54,544) is a probe reply. It reaches the healthy IoT devices too (3,049 replies). Fixing F1 should end it; if it does not, it is a separate problem.

Medium F5. Missing networks

Every AP should offer both networks. Missing for the full 30 min: NET-C on AP03, AP13, AP16, AP17; NET-T on AP06, AP13, AP28 (AP13 is not heard at all). The 8 devices "homed" on those APs (L-03, L-13, L-16, L-17, H-08, H-11, H-12, H-28) appear in 2,000–3,100 probe exchanges each and never join. Could be a disabled network or an AP on a channel no sensor covers: check the controller.

Medium F6. Slow beacons

All 914,057 beacons are marked 1 Mbit/s, a rate that does not exist on 5 GHz. If real, adverts alone use 10–19% of each channel (19% on ch 36/40 with 9 networks). At 6 Mbit/s it would be 1.7–3%. Plant rule: minimum basic rate 12–24 Mbit/s and at most 2–3 networks per AP.

Info

F7. What sensors can and cannot hear

Access-point replies outnumber device scans 29 to 1 (173,964 vs 6,024). Only 12 of 63 corporate devices and 4 of 24 IoT devices are heard during their own join exchange. Beacon loss is 1.7–2.3% everywhere and no sensor ever went deaf. So the tool must judge from the access point's side and never treat "not heard" as "did not happen". Example: one IoT join shows key message 1 but not 3, yet the AP sets up Block-Ack 60 ms later: connected, frame just not captured.

Healthy

F8–F9. Watch items and the good news

15 devices (laptops, phones, IoT) said "I'm leaving" within the first 9 s; one more at 12.4 min. On its own that is normal; as a burst it suggests a shared trigger. The tools network is healthy: 24 IoT devices, 31 handshakes reach M3 (M1→M3 in 0.3–2.5 ms), no key, MIC or group-key failures. No device roamed and no access point ever refused a knock or sign-in (all status codes 0).

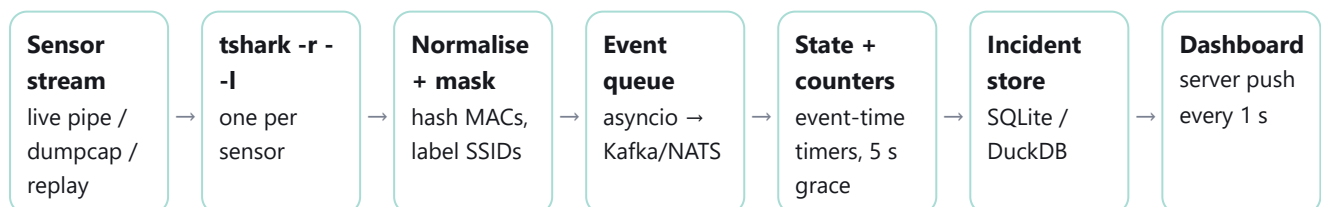
Several sensors, one story. Each sensor listens on its own channel and hears its own access points; no network is heard by two sensors, so frame-level de-duplication finds nothing here. The link between sensors is the pattern: the same failure signature on 26 access points across all 8 sensors = **one root incident** with 63 affected devices, not 63 or 654 incidents. Sensor start times differ by up to 2.3 s, so cross-sensor matching allows several seconds of slack.

4 Can it run in real time? Yes, and we measured it

We replayed the real recordings into eight live tshark pipes (one per sensor), paced by the original timestamps, and timed each frame from "written to the pipe" to "decoded and processed in Python".

Replay speed	Frames per second handled	Delay per frame, median / 99th percentile	Kept up?
1× (live pace, 60 s)	571 (the site's real rate)	2.8 ms / 10.5 ms	Yes
10× (10 min in 60 s)	6,681	1.5 ms / 8.3 ms	Yes
30× (30 min requested in 60 s)	9,552 (took 117 s)	2.2 ms / 9.9 ms	No, maximum ≈15× live

What limits "real time" is the Wi-Fi protocol, not the computer. A stall can only be called when the access point re-asks (30 s). With the rule "same stall on 3 access points", the site-wide incident appears at 47.5 s into the recording, **36 s after the first failing join**; the kicks confirm it at 77.5 s. Graphs and counters update every second.



Design choices for the live dashboard

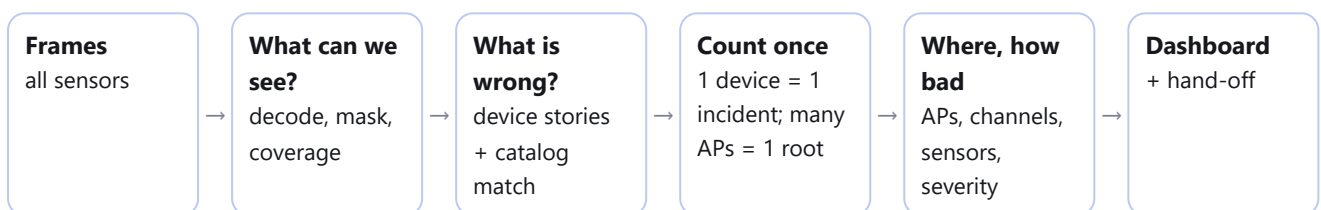
Choice	Why
tshark per sensor, reading a pipe with <code>-l</code>	Measured: 30× faster than scapy (7,700 vs 244 frames/s per core) and line-flushed, so each frame is available at once.

Event-time timers + watermark	"Nothing happened for 30 s" must be judged on capture time, not arrival time, or a slow sensor creates false stalls.
5 s grace window when merging sensors	Sensor clocks differ by up to 2.3 s.
Incident lifecycle open → updating → resolved	The dashboard shows live counts on one incident instead of adding new rows every minute.
FastAPI + Server-Sent Events + a light chart library	True push to the browser each second; Streamlit is quicker to build but re-runs the whole page on every refresh. For the demo, a replay mode feeds the recorded files at 1× or faster.

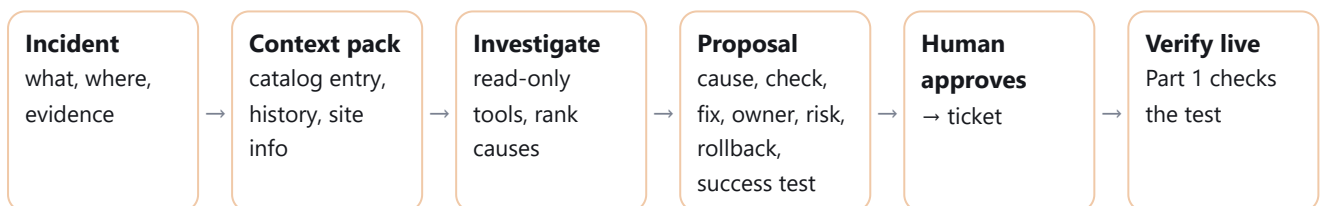
5 The three parts, at a glance

Full high-level and deep-level flows for every part are in [v3/flows.html](#) (styled) and [v3/flows.md](#) (Mermaid; all 8 diagrams render-tested).

Part 1: find WHAT is wrong and WHERE (real time)



Part 2: AI agent proposes HOW TO FIX



Example output for F1: "802.1X stalls after identity on 26 APs, fixed 30 s / 60 s timers, tools network healthy → check whether RADIUS receives requests from the controller. Owner: identity/AAA team. After the fix, clear the client-exclusion list for 18 laptops (F3). Success test: zero reason-23 kicks and > 95% logins succeeding for 10 minutes." Guardrails: the agent's tools are read-only, every claim cites evidence, codes are checked against the IEEE tables, identifiers stay masked, and below 60% confidence it asks a human for the missing fact.

Part 3: catalog every known error; learn the unknown ones



6 The error catalog (Part 3)

`catalog/error_catalog.json` (machine-readable) and `error_catalog.md` (readable) are generated by `catalog/build_catalog.py`. Each entry has: plain meaning, what it looks like in headers, detection rule and thresholds, the normal behaviour it must not be confused with, likely causes, how to confirm outside the headers, standard fix, owner team, related IEEE codes, and whether it appears in this data (with evidence).

79

catalog entries in 13 categories

63 / 63

802.11 reason codes mapped to an entry

120 / 139

status codes mapped; the other 19 are "Reserved" → OTHER

Category	Entries	Seen in data	Seen here (examples)
DISC Discovery and beacons	9	4	DISC-01 Network not broadcast on an AP radio; DISC-02 Access point silent; DISC-03 Probe storm
AUTH 802.11 authentication (open, SAE, FT)	6	0	
ASSOC Association and reassociation	10	0	
EAP 802.1X / EAP (enterprise login)	8	2	EAP-01 802.1X stall after EAP Identity; EAP-04 Kicked out: 802.1X authentication failed (reason 23)
KEY 4-way and group-key handshakes	6	1	KEY-06 Handshake step not observed (visibility gap)
SESS Disconnects and session stability	12	4	SESS-01 Join-fail-kick loop; SESS-02 Fast reject / client exclusion (reason 2 repeated); SESS-08 Mass leave burst
ROAM Roaming	4	0	
RF RF, airtime and congestion	7	3	RF-01 Missing acknowledgements / high retries; RF-03 Beacon overhead; RF-04 Basic rate too low / legacy rate
CLIENT Client behaviour and power save	2	0	
SEC Security signatures (passive detection only)	3	0	
INFRA Infrastructure-level causes inferred from the air	3	1	INFRA-01 Central authentication service outage
SENS Sensor and data quality	6	5	SENS-02 Sensor clock offset; SENS-03 Partial capture window; SENS-04 Physically impossible radio values
OTHER Unclassified: goes to the discovery agent	3	0	

The OTHER rule: anything no entry matches is kept, never dropped: an unmapped or reserved code (OTHER-CODE), a sequence the device-story machine does not define (OTHER-SEQ), or a metric jump no detector explains (OTHER-STAT). These go to `unknown_events.jsonl` with ± 10 s of context, and the discovery agent turns clusters of them into draft entries. Draft detection rules are written in a small JSON rule language (fields, time window, group-by, threshold) rather than free code, so they can be checked automatically and back-tested before a human approves them.

7 Reflection: how sure are we?

Statement	Confidence	What would change it
Corporate logins stall at the identity step for every device, all 30 min	High	Nothing in this data; measured on 654 attempts
The cause is on the network/login-server side, not the devices	High	RADIUS logs showing it answered (would point at the controller instead)
F3 is a controller client-exclusion policy	Medium	Controller config/logs; headers show the behaviour, not the setting
The probe storm is caused by the login failure	Medium	Storm continuing after logins are fixed
The 7 networks are really missing	Medium	Finding them on a channel no sensor monitors
Real-time is feasible on a laptop	High	Measured: 15× headroom, ~10 ms p99 delay
Airtime numbers are physically meaningful	Low	The data shows emulation signs (1 Mbit/s on 5 GHz, constant signal)

8 What to do next

1. Build Part 1 as a streaming pipeline on `json_full/` replay first, then on live pipes (skill file updated with the verified facts and the real-time design).
 2. Wire detectors to catalog ids so every incident has a name, a meaning and a standard fix from day one.
 3. Build the Part 2 agent with read-only tools over the incident store and catalog; start with F1 as the test case.
 4. Add the OTHER store now, even before the Part 3 agent exists, so unknown events are kept from the first run.
 5. Pitch opener: "0 of 654 corporate logins, tools network fine on the same radios, flagged in 36 seconds: it's the login server, not the Wi-Fi."
-

9 Open questions for you

1. Which LLM access will the event provide for Parts 2 and 3 (Claude API key, model limits)? This decides whether the agents run live or on demand.
 2. May the Part 2 agent read (not change) any system beyond the captures, such as a mock controller or RADIUS log, for the demo? If not, its "confirm" steps stay as instructions for humans.
 3. Is there a device or AP inventory (even a list of expected APs and SSIDs)? It would turn F5 from "medium" into "high" confidence.
-

A Files

<code>convert_full.py</code>	Full-capture converter (tshark, 102 s for all 8 files) → <code>json_full/*.jsonl</code>
<code>validate_converter.py</code>	Proves tshark output matches the scapy JSON on 28,536 frames
<code>verify_findings.py</code>	Recomputes every number in this report (36 checks)
<code>realtime_bench.py</code>	Live-pipe replay benchmark (speed, latency)

<code>catalog/</code>	IEEE code tables, catalog builder, <code>error_catalog.json/.md</code>
<code>v3/flows.html</code> , <code>v3/flows.md</code>	High- and deep-level flows for real time, Part 1, Part 2, Part 3
<code>.claude/skills/airframe-wifi-analysis/</code>	Build guide and test oracle for the coding phase

Proprietary and confidential. Built from the Airframe hackathon captures under NDA. Identifiers pseudonymised: L-nn laptops, H-nn phones/handhelds, I-nn IoT devices (nn = last address byte in decimal), APnn access points, NET-C corporate, NET-T tools.